MindWall — Production Architecture Specification

Version: 1.0.0
Classification: Internal Engineering Document
Project: MindWall — Cognitive Firewall / AI-Powered Human Manipulation Detection

1. System Overview

MindWall is a fully self-hosted, privacy-first cybersecurity platform that intercepts, analyzes, and scores incoming communications (email, browser-based webmail) for psychological manipulation tactics using a locally-run, fine-tuned large language model. Zero data leaves the deployment boundary. All inference runs on-premises on a 24GB VRAM GPU server.

2. High-Level Architecture Diagram

```
┌──────────────
```

```
│
│                          │                            │
│
│                          │  ┌──────────────────────┐  │
│
│                          │  │  SSL Termination Layer  │ │
│
│                          │  │  (upstream TLS handled) │ │
│
│                          │  └──────────┬───────────┘  │
│
│                          │             │              │
│
│                          │  ┌──────────▼───────────┐  │
│
│                          │  │   IMAP Command Parser   │ │
│
│                          │  │  (asyncio stream-based) │ │
│
│                          │  └──────────┬───────────┘  │
│
│                          │             │ FETCH intercept
│
│                          │  ┌──────────▼───────────┐  │
│
│                          │  │  Email Body Extractor   │ │
│
│                          │  │  (MIME parser + cleaner)│ │
│
│                          │  └──────────┬───────────┘  │
│
│                          └─────────────┼──────────────┘
│
│                                        │ HTTP POST (internal only)
│
│                                        │
│
│                          ┌─────────────▼──────────────┐
│
│                          │    FASTAPI CORE ENGINE       │
│
│                          │    (mindwall-api:8000)       │
│
│                          │                              │
│
│                          │  ┌──────────────────────┐   │
│
│                          │  │   Rule-Based Pre-Filter │  │
│
│                          │  │   (zero GPU, <5ms)      │  │
│
│                          │  └──────────┬───────────┘   │
│
```

```
│                         │               │ if passes     │
│
│                         │  ┌────────────▼────────────┐  │
│
│                         │  │  Behavioral Baseline    │  │
│
│                         │  │  Engine (SQLite)        │  │
│
│                         │  └─────────────┬───────────┘  │
│
│                         │               │              │
│
│                         │  ┌────────────▼────────────┐  │
│
│                         │  │   LLM Analysis Engine   │  │
│
│                         │  │   (Ollama HTTP client)  │  │
│
│                         │  └─────────────┬───────────┘  │
│
│                         │               │              │
│
│                         │  ┌────────────▼────────────┐  │
│
│                         │  │   Score Aggregator      │  │
│
│                         │  │   12-Dimension Scorer   │  │
│
│                         │  └─────────────┬───────────┘  │
│
│                         │               │              │
│
│                         │  ┌────────────▼────────────┐  │
│
│                         │  │   Alert & DB Writer     │  │
│
│                         │  │   (SQLite + WebSocket)  │  │
│
│                         │  └─────────────┬───────────┘  │
│
│                         └────────────────┼──────────────┘
│
│                                          │
│
│               ┌──────────────────────────┼──────────────────────┐
│
│               │                          │                      │
│
│  ┌────────────▼────────────┐  ┌────────────▼────────────┐
┌────────────▼────────────┐ │
│  │   OLLAMA LLM SERVER  │  │  REACT DASHBOARD     │  │  BROWSER EXTENSION
│ │
│  │  (mindwall-ollama)   │  │  (mindwall-ui:3000)  │  │  (Chrome/Firefox)
```

```
│ │
│  │                      │  │                      │  │
│ │
│  │  Llama 3.1 8B        │  │  Real-time Alerts    │  │  Gmail Web Intercept
│ │
│  │  Fine-tuned (LoRA)   │  │  WebSocket Feed      │  │  → localhost:8000
│ │
│  │  GGUF Q5_K_M         │  │  Threat Dashboard    │  │  /api/analyze
│ │
│  │  24GB GPU Server     │  │  Employee Risk View  │  │
│ │
│  └──────────────────────┘  └──────────────────────┘
└──────────────────────────┘ │
│
│
│  ┌──────────────────────────────────────────────────────────────────────┐
│
│  │                        UPSTREAM MAIL SERVERS
│  │
│  │          imap.gmail.com / outlook.office365.com / imap.yahoo.com
│  │
│  └──────────────────────────────────────────────────────────────────────┘
│
└──────────────────────────────────────────────────────────────────────────
────┘
```

---

3. Complete Repository Structure

```
mindwall/
│
├── docker-compose.yml                    # Orchestrates all services
├── docker-compose.override.yml           # Dev overrides (volume mounts, hot
reload)
├── .env.example                          # Environment variable template
├── setup.sh                              # One-command bootstrapper
├── Makefile                              # dev/prod task runner
│
├── proxy/                                # IMAP/SMTP Proxy Service
│   ├── Dockerfile
│   ├── requirements.txt
│   ├── main.py                           # Entrypoint — starts IMAP + SMTP
proxy
│   ├── imap/
│   │   ├── __init__.py
│   │   ├── server.py                     # asyncio IMAP server (listens
localhost:1143)
│   │   ├── upstream.py                   # asyncio connection to upstream IMAP
server
│   │   ├── parser.py                     # IMAP command/response parser (RFC
```

```
3501)
│   │   ├── interceptor.py                # FETCH interceptor — extracts email
bodies
│   │   └── injector.py                   # Risk score injection into
subject/header
│   ├── smtp/
│   │   ├── __init__.py
│   │   ├── server.py                     # asyncio SMTP server (listens
localhost:1025)
│   │   └── upstream.py                   # Forwards to real SMTP server
│   ├── mime/
│   │   ├── __init__.py
│   │   ├── parser.py                     # MIME email parser (text/html
extraction)
│   │   └── sanitizer.py                  # Strip HTML, normalize whitespace
│   ├── ssl/
│   │   ├── __init__.py
│   │   └── handler.py                    # TLS termination + upstream TLS
upgrade
│   └── config.py                         # Proxy config (upstream hosts,
ports)
│
├── api/                                  # FastAPI Core Engine Service
│   ├── Dockerfile
│   ├── requirements.txt
│   ├── main.py                           # FastAPI app factory
│   ├── core/
│   │   ├── __init__.py
│   │   ├── config.py                     # Settings via pydantic-settings
│   │   ├── lifespan.py                   # Startup/shutdown events
│   │   └── logging.py                    # Structured JSON logging (structlog)
│   ├── routers/
│   │   ├── __init__.py
│   │   ├── analyze.py                    # POST /api/analyze (from proxy +
extension)
│   │   ├── dashboard.py                  # GET /api/dashboard/* (stats,
threats)
│   │   ├── alerts.py                     # GET/PATCH /api/alerts/*
│   │   ├── employees.py                  # GET/POST /api/employees/*
│   │   ├── settings.py                   # GET/PUT /api/settings/*
│   │   └── websocket.py                  # WS /ws/alerts (real-time push)
│   ├── analysis/
│   │   ├── __init__.py
│   │   ├── pipeline.py                   # Orchestrates full analysis pipeline
│   │   ├── prefilter.py                  # Rule-based fast filter (regex +
keyword)
│   │   ├── llm_client.py                 # Ollama HTTP client (async httpx)
│   │   ├── prompt_builder.py             # Structured prompt construction
│   │   ├── scorer.py                     # 12-dimension score parser +
aggregator
│   │   ├── behavioral/
│   │   │   ├── __init__.py
│   │   │   ├── baseline.py               # Per-sender communication baseline
```

```
engine
│   │   │   ├── deviation.py              # Deviation scoring vs baseline
│   │   │   └── cross_channel.py          # Multi-channel coordination detector
│   │   └── dimensions.py                 # 12 manipulation dimension
definitions
│   ├── db/
│   │   ├── __init__.py
│   │   ├── database.py                   # SQLAlchemy async engine (aiosqlite)
│   │   ├── models.py                     # ORM models
│   │   ├── repositories/
│   │   │   ├── __init__.py
│   │   │   ├── alert_repo.py
│   │   │   ├── employee_repo.py
│   │   │   ├── baseline_repo.py
│   │   │   └── analysis_repo.py
│   │   └── migrations/
│   │       └── init_schema.sql           # Schema DDL
│   ├── schemas/
│   │   ├── __init__.py
│   │   ├── analyze.py                    # Request/response pydantic models
│   │   ├── alert.py
│   │   ├── dashboard.py
│   │   └── employee.py
│   ├── websocket/
│   │   ├── __init__.py
│   │   ├── manager.py                    # WebSocket connection manager
│   │   └── events.py                     # Event types + serializers
│   └── middleware/
│       ├── __init__.py
│       ├── auth.py                       # API key auth (internal network
only)
│       └── request_id.py                 # Request tracing middleware
│
├── finetune/                             # Run locally on 8GB GPU
│   ├── requirements.txt                  # unsloth, transformers, datasets,
etc.
│   ├── prepare_dataset.py                # Downloads + formats training data
│   ├── train.py                          # Unsloth QLoRA fine-tuning script
│   ├── evaluate.py                       # Evaluation against held-out test
set
│   ├── export.py                         # Merge LoRA + export to GGUF for
Ollama
│   ├── datasets/
│   │   ├── download.sh                   # Pulls public phishing/SE corpora
│   │   └── synthetic_generator.py        # Generates synthetic manipulation
examples
│   └── configs/
│       └── qlora_config.yaml             # Training hyperparameters
│
├── dashboard/                            # React + Tailwind Admin UI
│   ├── Dockerfile
│   ├── package.json
│   ├── tailwind.config.js
```

```
│   ├── vite.config.js
│   └── src/
│       ├── main.jsx
│       ├── App.jsx
│       ├── api/
│       │   ├── client.js                # Axios instance + interceptors
│       │   └── websocket.js             # WebSocket client manager
│       ├── components/
│       │   ├── layout/
│       │   │   ├── Sidebar.jsx
│       │   │   ├── TopBar.jsx
│       │   │   └── Layout.jsx
│       │   ├── alerts/
│       │   │   ├── AlertCard.jsx        # Individual threat alert card
│       │   │   ├── AlertFeed.jsx        # Real-time incoming alert feed
│       │   │   └── AlertDetail.jsx      # Full breakdown modal
│       │   ├── dashboard/
│       │   │   ├── ThreatGauge.jsx      # Org-wide threat level gauge
│       │   │   ├── DimensionRadar.jsx   # 12-dimension radar chart
│       │   │   ├── ThreatTimeline.jsx   # Historical threat graph
│       │   │   └── RiskHeatmap.jsx      # Employee risk heatmap
│       │   └── employees/
│       │       ├── EmployeeTable.jsx
│       │       └── EmployeeRiskProfile.jsx
│       └── pages/
│           ├── Dashboard.jsx
│           ├── Alerts.jsx
│           ├── Employees.jsx
│           └── Settings.jsx
│
├── extension/                           # Browser Extension (Gmail Web)
│   ├── manifest.json                    # WebExtension Manifest V3
│   ├── background.js                    # Service worker
│   ├── content_gmail.js                 # Gmail DOM observer + extractor
│   └── icons/
│
└── data/                                # Docker volume mount (persistent)
    ├── db/
    │   └── mindwall.db                  # SQLite database
    └── models/                          # Ollama model storage
```

---

4. Service Definitions — docker-compose.yml

```yaml
version: "3.9"

services:

  ollama:
    image: ollama/ollama:latest
```

```yaml
    container_name: mindwall-ollama
    restart: always
    volumes:
      - ./data/models:/root/.ollama
    environment:
      - OLLAMA_NUM_PARALLEL=4
      - OLLAMA_MAX_LOADED_MODELS=1
      - OLLAMA_FLASH_ATTENTION=1
    deploy:
      resources:
        reservations:
          devices:
            - driver: nvidia
              count: 1
              capabilities: [gpu]
    networks:
      - mindwall-internal
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:11434/api/tags"]
      interval: 30s
      timeout: 10s
      retries: 5
      start_period: 60s

  api:
    build:
      context: ./api
      dockerfile: Dockerfile
    container_name: mindwall-api
    restart: always
    depends_on:
      ollama:
        condition: service_healthy
    volumes:
      - ./data/db:/app/data/db
    environment:
      - OLLAMA_BASE_URL=http://ollama:11434
      - OLLAMA_MODEL=mindwall-llama3.1-8b
      - DATABASE_URL=sqlite+aiosqlite:////app/data/db/mindwall.db
      - API_SECRET_KEY=${API_SECRET_KEY}
      - LOG_LEVEL=INFO
      - WORKERS=4
    ports:
      - "8000:8000"
    networks:
      - mindwall-internal
      - mindwall-host
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8000/health"]
      interval: 15s
      timeout: 5s
      retries: 3
```

```yaml
  proxy:
    build:
      context: ./proxy
      dockerfile: Dockerfile
    container_name: mindwall-proxy
    restart: always
    depends_on:
      api:
        condition: service_healthy
    environment:
      - API_BASE_URL=http://api:8000
      - API_SECRET_KEY=${API_SECRET_KEY}
      - IMAP_LISTEN_HOST=0.0.0.0
      - IMAP_LISTEN_PORT=1143
      - SMTP_LISTEN_HOST=0.0.0.0
      - SMTP_LISTEN_PORT=1025
      - LOG_LEVEL=INFO
    ports:
      - "1143:1143"       # IMAP — email clients connect here
      - "1025:1025"       # SMTP — outbound (optional monitoring)
    networks:
      - mindwall-internal
      - mindwall-host

  dashboard:
    build:
      context: ./dashboard
      dockerfile: Dockerfile
    container_name: mindwall-ui
    restart: always
    depends_on:
      - api
    environment:
      - VITE_API_BASE_URL=http://localhost:8000
      - VITE_WS_URL=ws://localhost:8000/ws/alerts
    ports:
      - "3000:80"
    networks:
      - mindwall-host

networks:
  mindwall-internal:
    driver: bridge
    internal: true          # Ollama + API cannot reach internet directly
  mindwall-host:
    driver: bridge
```

---

5. Database Schema

```sql
-- migrations/init_schema.sql

CREATE TABLE IF NOT EXISTS employees (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    email           TEXT NOT NULL UNIQUE,
    display_name    TEXT,
    department      TEXT,
    risk_score      REAL DEFAULT 0.0,     -- rolling 30-day risk score
    created_at      DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at      DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS sender_baselines (
    id                  INTEGER PRIMARY KEY AUTOINCREMENT,
    recipient_email     TEXT NOT NULL,
    sender_email        TEXT NOT NULL,
    avg_word_count      REAL,
    avg_sentence_length REAL,
    typical_hours       TEXT,             -- JSON: [8,9,10,17,18] (typical send
hours)
    formality_score     REAL,             -- 0.0–1.0
    typical_requests    TEXT,             -- JSON: common request types
observed
    sample_count        INTEGER DEFAULT 0,
    last_updated        DATETIME DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(recipient_email, sender_email)
);

CREATE TABLE IF NOT EXISTS analyses (
    id                      INTEGER PRIMARY KEY AUTOINCREMENT,
    message_uid             TEXT NOT NULL,            -- IMAP UID or extension-
generated ID
    recipient_email         TEXT NOT NULL,
    sender_email            TEXT NOT NULL,
    sender_display_name     TEXT,
    subject                 TEXT,
    received_at             DATETIME,
    analyzed_at             DATETIME DEFAULT CURRENT_TIMESTAMP,
    channel                 TEXT NOT NULL,            -- 'imap' | 'gmail_web'
    prefilter_triggered     BOOLEAN DEFAULT FALSE,
    prefilter_signals       TEXT,                     -- JSON: list of
triggered rules
    manipulation_score      REAL,                     -- 0–100 aggregate
    dimension_scores        TEXT,                     -- JSON: {dimension:
score, ...}
    explanation             TEXT,                     -- LLM plain-English
explanation
    recommended_action      TEXT,                     -- 'proceed' | 'verify' |
'block'
    llm_raw_response        TEXT,                     -- Full LLM JSON output
(stored for audit)
    processing_time_ms      INTEGER,
```

```
    UNIQUE(message_uid, recipient_email)
);

CREATE TABLE IF NOT EXISTS alerts (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    analysis_id     INTEGER NOT NULL REFERENCES analyses(id),
    severity        TEXT NOT NULL,        -- 'low' | 'medium' | 'high' |
'critical'
    acknowledged    BOOLEAN DEFAULT FALSE,
    acknowledged_by TEXT,
    acknowledged_at DATETIME,
    created_at      DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_analyses_recipient   ON analyses(recipient_email, analyzed_at
DESC);
CREATE INDEX idx_analyses_score       ON analyses(manipulation_score DESC);
CREATE INDEX idx_alerts_severity      ON alerts(severity, acknowledged,
created_at DESC);
CREATE INDEX idx_baselines_lookup     ON sender_baselines(recipient_email,
sender_email);
```

---

6. Core Analysis Pipeline

6.1 — 12 Manipulation Dimensions

```
# api/analysis/dimensions.py

from dataclasses import dataclass
from enum import Enum

class Dimension(Enum):
    ARTIFICIAL_URGENCY          = "artificial_urgency"
    AUTHORITY_IMPERSONATION     = "authority_impersonation"
    FEAR_THREAT_INDUCTION       = "fear_threat_induction"
    RECIPROCITY_EXPLOITATION    = "reciprocity_exploitation"
    SCARCITY_TACTICS            = "scarcity_tactics"
    SOCIAL_PROOF_MANIPULATION   = "social_proof_manipulation"
    SENDER_BEHAVIORAL_DEVIATION = "sender_behavioral_deviation"
    CROSS_CHANNEL_COORDINATION  = "cross_channel_coordination"
    EMOTIONAL_ESCALATION        = "emotional_escalation"
    REQUEST_CONTEXT_MISMATCH    = "request_context_mismatch"
    UNUSUAL_ACTION_REQUESTED    = "unusual_action_requested"
    TIMING_ANOMALY              = "timing_anomaly"

DIMENSION_WEIGHTS = {
    Dimension.ARTIFICIAL_URGENCY:          0.12,
    Dimension.AUTHORITY_IMPERSONATION:     0.15,
    Dimension.FEAR_THREAT_INDUCTION:       0.12,
```

```
    Dimension.RECIPROCITY_EXPLOITATION:    0.07,
    Dimension.SCARCITY_TACTICS:            0.07,
    Dimension.SOCIAL_PROOF_MANIPULATION:   0.06,
    Dimension.SENDER_BEHAVIORAL_DEVIATION: 0.12,
    Dimension.CROSS_CHANNEL_COORDINATION:  0.08,
    Dimension.EMOTIONAL_ESCALATION:        0.07,
    Dimension.REQUEST_CONTEXT_MISMATCH:    0.06,
    Dimension.UNUSUAL_ACTION_REQUESTED:    0.05,
    Dimension.TIMING_ANOMALY:              0.03,
}
```

6.2 — LLM Prompt Construction

```
# api/analysis/prompt_builder.py

SYSTEM_PROMPT = """
You are MindWall, a cybersecurity analysis engine specialized in detecting
psychological manipulation tactics in business communications. You analyze
emails and messages with clinical precision, identifying social engineering
patterns used by attackers to manipulate recipients into unsafe actions.

You always respond with a valid JSON object and nothing else.
""".strip()

def build_analysis_prompt(
    email_body: str,
    sender_email: str,
    sender_display_name: str,
    subject: str,
    received_hour: int,
    baseline: dict | None,
    prefilter_signals: list[str],
) -> str:
    baseline_context = ""
    if baseline:
        baseline_context = f"""
SENDER BEHAVIORAL BASELINE (historical communication pattern):
- Average word count per email: {baseline['avg_word_count']:.0f}
- Average sentence length: {baseline['avg_sentence_length']:.1f} words
- Typical send hours (UTC): {baseline['typical_hours']}
- Formality score (0=casual, 1=formal): {baseline['formality_score']:.2f}
- This email's send hour: {received_hour}
- Word count deviation: {baseline.get('word_count_deviation', 'N/A')}
"""

    prefilter_context = ""
    if prefilter_signals:
        prefilter_context = f"\nFAST-FILTER PRE-SIGNALS DETECTED: {',
'.join(prefilter_signals)}"
```

```
    return f"""
Analyze the following email for psychological manipulation tactics.
{prefilter_context}
{baseline_context}

EMAIL METADATA:
- Sender: {sender_display_name} <{sender_email}>
- Subject: {subject}
- Received Hour (UTC): {received_hour}

EMAIL BODY:
---
{email_body[:4000]}
---

Score each of the following 12 manipulation dimensions from 0 to 100:
- artificial_urgency: manufactured time pressure or deadline
- authority_impersonation: falsely claiming or implying authority
- fear_threat_induction: using threats, consequences, or fear
- reciprocity_exploitation: leveraging past favors or obligations
- scarcity_tactics: creating false scarcity of time, resource, or opportunity
- social_proof_manipulation: fabricating consensus or peer behavior
- sender_behavioral_deviation: deviation from this sender's typical
communication style
- cross_channel_coordination: evidence of coordinated multi-channel attack
- emotional_escalation: escalating emotional intensity to override rational
thinking
- request_context_mismatch: the request is inconsistent with the stated context
- unusual_action_requested: requesting actions atypical for legitimate business
communication
- timing_anomaly: suspicious timing relative to sender's typical patterns

Respond ONLY with this JSON structure:
{{
    "dimension_scores": {{
        "artificial_urgency": <0-100>,
        "authority_impersonation": <0-100>,
        "fear_threat_induction": <0-100>,
        "reciprocity_exploitation": <0-100>,
        "scarcity_tactics": <0-100>,
        "social_proof_manipulation": <0-100>,
        "sender_behavioral_deviation": <0-100>,
        "cross_channel_coordination": <0-100>,
        "emotional_escalation": <0-100>,
        "request_context_mismatch": <0-100>,
        "unusual_action_requested": <0-100>,
        "timing_anomaly": <0-100>
    }},
    "primary_tactic": "<name of highest-scoring dimension>",
    "explanation": "<1-2 sentence plain English explanation of what
manipulation is occurring, written to warn a non-technical employee>",
    "recommended_action": "<proceed|verify|block>",
    "confidence": <0-100>
```

```
}}
"""
```

6.3 — Analysis Pipeline Orchestrator

```python
# api/analysis/pipeline.py

import time
import json
import asyncio
from datetime import datetime, timezone

from .prefilter import PreFilter
from .llm_client import OllamaClient
from .prompt_builder import build_analysis_prompt, SYSTEM_PROMPT
from .scorer import ScoreAggregator
from .behavioral.baseline import BaselineEngine
from .behavioral.deviation import DeviationScorer
from ..db.repositories.analysis_repo import AnalysisRepository
from ..db.repositories.alert_repo import AlertRepository
from ..db.repositories.baseline_repo import BaselineRepository
from ..websocket.manager import WebSocketManager
from ..schemas.analyze import AnalyzeRequest, AnalyzeResponse


class AnalysisPipeline:
    def __init__(
        self,
        llm: OllamaClient,
        analysis_repo: AnalysisRepository,
        alert_repo: AlertRepository,
        baseline_repo: BaselineRepository,
        ws_manager: WebSocketManager,
    ):
        self.prefilter = PreFilter()
        self.llm = llm
        self.aggregator = ScoreAggregator()
        self.baseline_engine = BaselineEngine(baseline_repo)
        self.deviation_scorer = DeviationScorer()
        self.analysis_repo = analysis_repo
        self.alert_repo = alert_repo
        self.ws_manager = ws_manager

    async def run(self, request: AnalyzeRequest) -> AnalyzeResponse:
        start_time = time.monotonic()

        # Stage 1: Rule-based prefilter (no GPU, <5ms)
        prefilter_result = self.prefilter.evaluate(
            subject=request.subject,
            body=request.body,
```

```
            sender_email=request.sender_email,
            received_at=request.received_at,
        )

        # Stage 2: Load sender behavioral baseline
        baseline = await self.baseline_engine.get_baseline(
            recipient_email=request.recipient_email,
            sender_email=request.sender_email,
        )

        # Stage 3: Compute behavioral deviation scores
        deviation_context = self.deviation_scorer.score(
            body=request.body,
            received_at=request.received_at,
            baseline=baseline,
        )

        # Stage 4: Build prompt and call LLM
        prompt = build_analysis_prompt(
            email_body=request.body,
            sender_email=request.sender_email,
            sender_display_name=request.sender_display_name,
            subject=request.subject,
            received_hour=request.received_at.hour if request.received_at else
datetime.now(timezone.utc).hour,
            baseline=baseline,
            prefilter_signals=prefilter_result.signals,
        )

        llm_response_raw = await self.llm.generate(
            system_prompt=SYSTEM_PROMPT,
            user_prompt=prompt,
        )

        llm_data = json.loads(llm_response_raw)

        # Stage 5: Merge LLM scores with behavioral deviation scores
        final_scores = self.aggregator.merge(
            llm_dimension_scores=llm_data["dimension_scores"],
            behavioral_deviation_score=deviation_context.deviation_score,
        )
        aggregate_score = self.aggregator.compute_aggregate(final_scores)

        # Stage 6: Determine severity
        severity = self._severity(aggregate_score)

        processing_ms = int((time.monotonic() - start_time) * 1000)

        # Stage 7: Persist analysis record
        analysis_id = await self.analysis_repo.insert(
            message_uid=request.message_uid,
            recipient_email=request.recipient_email,
            sender_email=request.sender_email,
```

```
            sender_display_name=request.sender_display_name,
            subject=request.subject,
            received_at=request.received_at,
            channel=request.channel,
            prefilter_triggered=prefilter_result.triggered,
            prefilter_signals=prefilter_result.signals,
            manipulation_score=aggregate_score,
            dimension_scores=final_scores,
            explanation=llm_data["explanation"],
            recommended_action=llm_data["recommended_action"],
            llm_raw_response=llm_response_raw,
            processing_time_ms=processing_ms,
        )

        # Stage 8: Create alert if above threshold
        if aggregate_score >= 35:
            alert_id = await self.alert_repo.insert(
                analysis_id=analysis_id,
                severity=severity,
            )
            # Stage 9: Push real-time alert to dashboard
            await self.ws_manager.broadcast({
                "event": "new_alert",
                "alert_id": alert_id,
                "analysis_id": analysis_id,
                "recipient_email": request.recipient_email,
                "sender_email": request.sender_email,
                "subject": request.subject,
                "manipulation_score": aggregate_score,
                "severity": severity,
                "explanation": llm_data["explanation"],
                "recommended_action": llm_data["recommended_action"],
                "dimension_scores": final_scores,
            })

        # Stage 10: Update sender baseline asynchronously
        asyncio.create_task(
            self.baseline_engine.update_baseline(
                recipient_email=request.recipient_email,
                sender_email=request.sender_email,
                body=request.body,
                received_at=request.received_at,
            )
        )

        return AnalyzeResponse(
            analysis_id=analysis_id,
            manipulation_score=aggregate_score,
            severity=severity,
            explanation=llm_data["explanation"],
            recommended_action=llm_data["recommended_action"],
            dimension_scores=final_scores,
            processing_time_ms=processing_ms,
```

```
        )

    @staticmethod
    def _severity(score: float) -> str:
        if score >= 80:  return "critical"
        if score >= 60:  return "high"
        if score >= 35:  return "medium"
        return "low"
```

---

7. IMAP Proxy Architecture

```
# proxy/imap/server.py

import asyncio
import ssl
from .upstream import UpstreamIMAPConnection
from .interceptor import FetchInterceptor
from .injector import RiskScoreInjector

class MindWallIMAPServer:
    """
    Transparent IMAP proxy that:
    1. Accepts connections from email clients on localhost:1143
    2. Opens authenticated connection to upstream IMAP server
    3. Intercepts FETCH responses containing email bodies
    4. Sends body to MindWall API for analysis
    5. Injects risk score into subject line before returning to client
    """

    def __init__(self, config: ProxyConfig):
        self.config = config
        self.interceptor = FetchInterceptor(config.api_base_url,
config.api_secret_key)
        self.injector = RiskScoreInjector()

    async def handle_client(
        self,
        client_reader: asyncio.StreamReader,
        client_writer: asyncio.StreamWriter,
    ):
        peer = client_writer.get_extra_info("peername")
        upstream = UpstreamIMAPConnection(
            host=self.config.upstream_imap_host,
            port=self.config.upstream_imap_port,
            use_ssl=True,
        )
        await upstream.connect()

        try:
```

```
            await self._pipe(client_reader, client_writer, upstream)
        finally:
            upstream.close()
            client_writer.close()

    async def _pipe(self, client_reader, client_writer, upstream):
        """
        Bidirectional pipe between client and upstream.
        Intercepts FETCH responses at the data level.
        """
        client_to_upstream = asyncio.create_task(
            self._forward_client_commands(client_reader, upstream)
        )
        upstream_to_client = asyncio.create_task(
            self._forward_upstream_responses(upstream, client_writer)
        )
        done, pending = await asyncio.wait(
            [client_to_upstream, upstream_to_client],
            return_when=asyncio.FIRST_COMPLETED,
        )
        for task in pending:
            task.cancel()

    async def _forward_upstream_responses(self, upstream, client_writer):
        """
        Reads responses from upstream IMAP.
        Detects FETCH responses containing RFC822/BODY content.
        Passes through interceptor before writing to client.
        """
        async for line in upstream.read_lines():
            processed = await self.interceptor.process_line(line)
            client_writer.write(processed)
            await client_writer.drain()

    async def start(self):
        server = await asyncio.start_server(
            self.handle_client,
            host=self.config.imap_listen_host,
            port=self.config.imap_listen_port,
        )
        async with server:
            await server.serve_forever()
```

---

8. Fine-Tuning Architecture (8GB GPU)

```
# finetune/train.py

from unsloth import FastLanguageModel
from trl import SFTTrainer
```

```python
from transformers import TrainingArguments
from datasets import load_from_disk

MAX_SEQ_LENGTH = 2048
MODEL_NAME = "unsloth/Meta-Llama-3.1-8B"
OUTPUT_DIR = "./output/mindwall-lora"

model, tokenizer = FastLanguageModel.from_pretrained(
    model_name=MODEL_NAME,
    max_seq_length=MAX_SEQ_LENGTH,
    dtype=None,          # auto-detect (bfloat16 on Ampere+)
    load_in_4bit=True,   # QLoRA — fits 8GB
)

model = FastLanguageModel.get_peft_model(
    model,
    r=16,                    # LoRA rank
    target_modules=[
        "q_proj", "k_proj", "v_proj", "o_proj",
        "gate_proj", "up_proj", "down_proj",
    ],
    lora_alpha=32,
    lora_dropout=0.05,
    bias="none",
    use_gradient_checkpointing="unsloth",  # 30% VRAM reduction
    random_state=42,
)

dataset = load_from_disk("./datasets/processed/mindwall_train")

trainer = SFTTrainer(
    model=model,
    tokenizer=tokenizer,
    train_dataset=dataset,
    dataset_text_field="text",
    max_seq_length=MAX_SEQ_LENGTH,
    args=TrainingArguments(
        per_device_train_batch_size=2,
        gradient_accumulation_steps=8,     # effective batch = 16
        warmup_steps=100,
        num_train_epochs=3,
        learning_rate=2e-4,
        fp16=False,
        bf16=True,
        logging_steps=25,
        optim="adamw_8bit",
        weight_decay=0.01,
        lr_scheduler_type="cosine",
        output_dir=OUTPUT_DIR,
        save_strategy="epoch",
    ),
)
```

```
trainer.train()
model.save_pretrained_merged(
    "./output/mindwall-merged",
    tokenizer,
    save_method="merged_16bit",
)
```

Training Data Sources (all public domain / permissive license)

Dataset	Source	Size	Content
PhishTank	phishtank.com	~1.5M	Known phishing URLs + content
CEAS 2008	Spam corpus	~30K	Phishing emails labeled
Enron Phishing	CMU	~1.7K	Annotated phishing in enterprise context
CSIRO Social Engineering	research.csiro.au	~500	Social engineering transcripts
Synthetic (generated)	synthetic_generator.py	~20K	GPT-4 generated labeled examples

9. Browser Extension (Gmail Web)

```
// extension/manifest.json
{
  "manifest_version": 3,
  "name": "MindWall",
  "version": "1.0.0",
  "description": "Real-time manipulation detection for Gmail",
  "permissions": ["activeTab", "scripting"],
  "host_permissions": ["https://mail.google.com/*"],
  "content_scripts": [
    {
      "matches": ["https://mail.google.com/*"],
      "js": ["content_gmail.js"],
      "run_at": "document_idle"
    }
  ],
  "background": {
    "service_worker": "background.js"
  }
}
```

The content script uses a `MutationObserver` to detect when Gmail renders an email body in the DOM, extracts the sender, subject, and body text, and POSTs to `http://localhost:8000/api/analyze`. On

receiving the response, it injects a colored risk badge directly into the Gmail message header DOM element.

---

10. Environment Variables

```
# .env.example

# API
API_SECRET_KEY=<generate with: openssl rand -hex 32>
DATABASE_URL=sqlite+aiosqlite:////app/data/db/mindwall.db
OLLAMA_BASE_URL=http://ollama:11434
OLLAMA_MODEL=mindwall-llama3.1-8b
OLLAMA_TIMEOUT_SECONDS=30
LOG_LEVEL=INFO
WORKERS=4

# Proxy
IMAP_LISTEN_HOST=0.0.0.0
IMAP_LISTEN_PORT=1143
SMTP_LISTEN_HOST=0.0.0.0
SMTP_LISTEN_PORT=1025

# Alert Thresholds
ALERT_MEDIUM_THRESHOLD=35
ALERT_HIGH_THRESHOLD=60
ALERT_CRITICAL_THRESHOLD=80
```

---

11. API Endpoints Reference

Method	Path	Description
`POST`	`/api/analyze`	Submit email for analysis (proxy + extension)
`GET`	`/api/dashboard/summary`	Org-wide threat summary stats
`GET`	`/api/dashboard/timeline`	Threat score timeline (with date range)
`GET`	`/api/alerts`	Paginated alert list with filters
`GET`	`/api/alerts/{id}`	Full alert detail with dimension breakdown
`PATCH`	`/api/alerts/{id}/acknowledge`	Mark alert as acknowledged
`GET`	`/api/employees`	Employee list with rolling risk scores
`GET`	`/api/employees/{email}/risk-profile`	Full risk profile + sender baselines

Method	Path	Description
GET	/api/settings	Current system settings
PUT	/api/settings	Update thresholds, upstream IMAP configs
GET	/health	Service health (used by Docker healthcheck)
WS	/ws/alerts	Real-time WebSocket feed for dashboard

12. Security Considerations

- The API listens on `0.0.0.0:8000` but should be firewalled to LAN only at the network level
- All inter-service communication is on the internal Docker bridge network — Ollama is never exposed externally
- The IMAP proxy never stores email content — only the extracted plain-text body is sent to the API, which stores only the LLM explanation and scores, never the raw email body
- API authentication uses a pre-shared secret key passed via `X-MindWall-Key` header — all internal services use this same key
- The SQLite database is stored on a host-mounted volume for persistence and backup compatibility
- SSL certificates for upstream IMAP connections are validated against system trust store — no SSL stripping occurs upstream

13. One-Command Setup

```
#!/usr/bin/env bash
# setup.sh

set -euo pipefail

echo "[MindWall] Checking dependencies..."
command -v docker >/dev/null 2>&1 || { echo "Docker required."; exit 1; }
command -v docker compose >/dev/null 2>&1 || { echo "Docker Compose required."; exit 1; }
nvidia-smi >/dev/null 2>&1 || { echo "NVIDIA GPU + drivers required."; exit 1; }

echo "[MindWall] Generating secrets..."
cp -n .env.example .env
SECRET=$(openssl rand -hex 32)
sed -i "s|API_SECRET_KEY=.*|API_SECRET_KEY=${SECRET}|" .env

echo "[MindWall] Creating data directories..."
mkdir -p data/db data/models

echo "[MindWall] Building and starting services..."
docker compose up -d --build

echo "[MindWall] Pulling LLM model (this may take a few minutes)..."
```

```
docker compose exec ollama ollama pull llama3.1:8b

echo ""
echo "✅ MindWall is running."
echo "   Dashboard:      http://localhost:3000"
echo "   API:            http://localhost:8000"
echo "   IMAP Proxy:     localhost:1143"
echo ""
echo "   Point your email client's IMAP server to: localhost:1143"
```